

**LAPORAN PRAKTIKUM**  
**ALGORITMA PEMROGRAMAN 2**  
**MODUL 12 – SEARCHING**



**RASYA DIKA PRATAMA**

**109082500006**

**KELAS S1IF-13-06**

**PROGRAM STUDI TEKNIK INFORMATIKA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

## A. Dasar Teori

Searching atau pencarian data adalah proses algoritma untuk menemukan nilai spesifik di dalam sekumpulan data, seperti *array* atau *struct*. Pada modul ini, terdapat dua metode utama yang digunakan yaitu *Sequential Search* dan *Binary Search*. *Sequential Search* bekerja dengan mengecek elemen satu per satu dari indeks awal hingga akhir, sehingga sangat fleksibel untuk sekumpulan data yang acak (belum terurut). Sebaliknya, *Binary Search* menawarkan performa yang jauh lebih cepat untuk jumlah data yang masif dengan membagi rentang indeks pencarian menjadi dua bagian pada setiap iterasinya, namun metode ini memiliki syarat wajib yaitu array data harus sudah dalam keadaan terurut (*ascending* atau *descending*) sebelum pencarian dieksekusi.

## B. Guided

### 1. Program Unguided 1

```
package main

import "fmt"

type arrData [5]string //tipe data alias

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1

    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }

    return found
}
```

```
}

func main() {

    var arrBinatang arrData //array tipe string

    for i := 0; i < len(arrBinatang); i++ {

        fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)

        fmt.Scan(&arrBinatang[i])

    }

    fmt.Println()

    var binatangCari string

    fmt.Print("Masukkan nama binatang yang mau dicari : ")

    fmt.Scan(&binatangCari)

    var idxCari int

    idxCari = seqSearch(arrBinatang, binatangCari)

    if idxCari > -1 {

        fmt.Printf("Data %s ditemukan pada indeks ke-%d!",
binatangCari, idxCari)

    } else if idxCari == -1 {

        fmt.Printf("Data %s tidak ditemukan!", binatangCari)

    }

}
```

Output :

```
dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_11\Guided\Guided-1
$ go run .\sequentialsearch.go
Masukkan data binatang indeks ke-0 : kucing
Masukkan data binatang indeks ke-1 : anjing
Masukkan data binatang indeks ke-2 : sapi
Masukkan data binatang indeks ke-3 : kambing
Masukkan data binatang indeks ke-4 : ayam

Masukkan nama binatang yang mau dicari : kambing
Data kambing ditemukan pada indeks ke-3!
```

Penjelasan:

Program ini merupakan implementasi dasar dari algoritma *Sequential Search* untuk mencari nama binatang di dalam sebuah *array* bertipe string.

Pencarian dilakukan dengan menggunakan perulangan *for* dari indeks awal hingga indeks terakhir *array*, lalu membandingkan setiap elemennya dengan kata kunci yang diinputkan *user*. Jika data cocok, program akan menyimpan posisi indeks tersebut ke dalam sebuah variabel dan langsung menghentikan perulangan menggunakan *statement break*, lalu menampilkan hasilnya ke layar.

## 2. Program guided 2

```
package main

import "fmt"

// fungsi binary search
func binarySearch(data []int, x int) int {

    // batas kiri array
    kiri := 0

    // batas kanan array
    kanan := len(data) - 1

    // perulangan selama kiri <= kanan
    for kiri <= kanan {
```

```

        // mencari index tengah
        tengah := (kiri + kanan) / 2

        // jika data ditemukan
        if data[tengah] == x {
            return tengah

        // jika x lebih besar, cari ke kanan
        } else if data[tengah] < x {
            kiri = tengah + 1

        // jika x lebih kecil, cari ke kiri
        } else {
            kanan = tengah - 1
        }
    }

    // jika data tidak ditemukan
    return -1
}

func main() {

    var n int
    var cari int

    // input jumlah data
    fmt.Print("Jumlah data: ")
    fmt.Scan(&n)

    // membuat array sesuai jumlah data
    data := make([]int, n)

    // input isi array
    fmt.Println("Masukkan data terurut:")
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

```

```

// input angka yang dicari
fmt.Print("Angka yang dicari: ")
fmt.Scan(&cari)

// memanggil fungsi binary search
hasil := binarySearch(data, cari)

// output hasil pencarian
if hasil != -1 {
    fmt.Println("Data ditemukan di index:", hasil)
} else {
    fmt.Println("Data tidak ditemukan")
}
}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_11\Guided\Guided-2
$ go run .\binarysearch.go
Jumlah data: 7
Masukkan data terurut:
1 3 5 7 9 11 13
Angka yang dicari: 9
Data ditemukan di index: 4

```

Penjelasan:

Program ini mengimplementasikan algoritma *Binary Search* untuk mencari angka di dalam *slice* bertipe *integer* yang diasumsikan sudah terurut. Logika utamanya berjalan dengan mencari nilai tengah (*median*) dari batas kiri dan batas kanan *array*. Jika angka yang dicari lebih besar dari elemen tengah, maka batas kiri akan digeser ke indeks tengah + 1, dan sebaliknya batas kanan digeser jika angka lebih kecil, perulangan ini terus berlanjut hingga angka ditemukan atau batas indeks saling bersilangan yang menandakan data tidak ada.

### 3. Program guided 3

```

package main

import "fmt"

```

```
type mahasiswa struct {
    nama string
    nim    int
    IPK    float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim {
            kn = med - 1
        } else if nimDicari > arrData[med].nim {
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
```

```

var nimDicari int

fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT
---")
for i := 0; i < len(mahasiswa06); i++ {
    fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks
ke-%d ===\n", i)
    fmt.Print("Masukkan nama : ")
    fmt.Scan(&mahasiswa06[i].nama)
    fmt.Print("Masukkan NIM : ")
    fmt.Scan(&mahasiswa06[i].nim)
    fmt.Print("Masukkan IPK : ")
    fmt.Scan(&mahasiswa06[i].IPK)

fmt.Println("-----")
}
fmt.Println()

fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
fmt.Scan(&nimDicari)
fmt.Println()

var idxHasilCariSeqSearch int
idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06,
nimDicari)

fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
if idxHasilCariSeqSearch > -1 {
    fmt.Printf("Data NIM mahasiswa %d ditemukan pada array
di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
    fmt.Println("Berikut Detail Datanya : ")
    fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
    fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
    fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
} else if idxHasilCariSeqSearch == -1 {

```



```

        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada
array!", nimDicari)
    }
    fmt.Println()

    var idxHasilCariBinSearch int
    idxHasilCariBinSearch = binSearchStruct(mahasiswa06,
nimDicari)

    fmt.Println("== HASIL BINARY SEARCH ==")
    if idxHasilCariBinSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada array
di indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada
array!", nimDicari)
    }
}

```

Output :

```

Masukkan NIM : 10903889203
Masukkan IPK : 3.9
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : rafa
Masukkan NIM : 10909283638
Masukkan IPK : 3.7
-----

Masukkan NIM mahasiswa yang ingin dicari : 10903889203

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 10903889203 ditemukan pada array di indeks ke-3!
Berikut Detail Datanya :
Nama : cherry
NIM : 10903889203
IPK : 3.90

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 10903889203 tidak ditemukan pada array!

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_11\Guided\Guided-3
$

```

Penjelasan:

Program ini bertujuan membandingkan penggunaan algoritma *Sequential Search* dan *Binary Search* pada tipe data bentukan (*array of struct*) yang menyimpan data mahasiswa. Pencarian data difokuskan pada *field* NIM mahasiswa. Untuk fungsi *Sequential Search*, pengecekan dilakukan berurutan tanpa mempedulikan susunan NIM, sedangkan untuk fungsi *Binary Search*, data awal harus diinputkan secara terurut berdasarkan NIM agar algoritma pembagian rentang indeks pencarian (kiri, kanan, median) dapat berfungsi secara logis dan mengembalikan indeks secara akurat.

### C. Unguided

#### 1. Soal Unguided 1

- 1) Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program **pilkart** yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

**Masukan** hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

**Keluaran** dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara.

| No | Masukan                    | Keluaran                                                                          |
|----|----------------------------|-----------------------------------------------------------------------------------|
| 1  | 7 19 3 2 78 3 1 -3 18 19 0 | Suara masuk: 10<br>Suara sah: 8<br>1: 1<br>2: 1<br>3: 2<br>7: 1<br>18: 1<br>19: 2 |

Jawaban :

```
package main

import "fmt"

func main() {

    var suara [21]int

    var masuk, sah, num int
```

```
for {  
    fmt.Scan(&num)  
  
    if num == 0 {  
        break  
    }  
  
    masuk++  
  
    if num >= 1 && num <= 20 {  
        sah++  
        suara[num]++  
    }  
}  
  
fmt.Printf("Suara masuk: %d\n", masuk)  
fmt.Printf("Suara sah: %d\n", sah)  
  
for i := 1; i <= 20; i++ {  
    if suara[i] > 0 {  
        fmt.Printf("%d: %d\n", i, suara[i])  
    }  
}  
}
```

Output :

```
dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_11\Unguided\Unguided-1
$ go run .\pemilihanrt.go
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
```

Penjelasan :

Program ini dirancang untuk menghitung hasil pemungutan suara dengan memvalidasi input dinamis menggunakan algoritma *array counting*. Alih-alih melakukan *searching* secara manual tiap angka, program memanfaatkan *array* suara dengan indeks 1 hingga 20 sebagai representasi nomor urut calon, di mana setiap kali input suara bernilai valid dimasukkan, elemen pada indeks tersebut akan langsung ditambahkan (di-*increment*). Proses pembacaan input (menggunakan *infinite loop*) akan otomatis berhenti saat menerima angka 0, lalu menampilkan rekapitulasi data yang tersimpan .

## 2. Soal Unguided 2

- 2) Berdasarkan program sebelumnya, buat program **pilkart** yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang

sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

**Masukan** hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

**Keluaran** dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

| No | Masukan                    | Keluaran                                                          |
|----|----------------------------|-------------------------------------------------------------------|
| 1  | 7 19 3 2 78 3 1 -3 18 19 0 | Suara masuk: 10<br>Suara sah: 8<br>Ketua RT: 3<br>Wakil ketua: 19 |

Jawaban :

```
package main

import "fmt"

func main() {
```

```
var suara [21]int

var masuk, sah, num int

for {

    fmt.Scan(&num)

    if num == 0 {

        break

    }

    masuk++

    if num >= 1 && num <= 20 {

        sah++

        suara[num]++

    }

}

fmt.Printf("Suara masuk: %d\n", masuk)

fmt.Printf("Suara sah: %d\n", sah)

var ketua, wakil int

for i := 1; i <= 20; i++ {

    if suara[i] > suara[ketua] {

        wakil = ketua

        ketua = i

    } else if suara[i] > suara[wakil] {

        wakil = i

    }

}
```

```

    }

    }

    fmt.Printf("Ketua RT: %d\n", ketua)

    fmt.Printf("Wakil ketua: %d\n", wakil)

}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_11\Unguided\Unguided-2
$ go run .\ketua-wakil.go
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19

```

Penjelasan :

Melanjutkan fungsionalitas dari program perhitungan suara sebelumnya, program ini menambahkan algoritma untuk mencari nilai perolehan suara terbanyak pertama (sebagai Ketua RT) dan terbanyak kedua (sebagai Wakil Ketua RT). Pencarian nilai maksimal ini dilakukan melalui iterasi *array* suara secara sekuensial, di mana jika ditemukan perolehan kandidat yang melebihi perolehan ketua saat ini, maka posisi ketua otomatis turun menjadi wakil, dan kandidat baru tersebut mengisi posisi ketua. Karena iterasi mengecek dari indeks terkecil (1) ke terbesar (20), syarat *tie-breaker* untuk mendahulukan nomor urut terkecil juga langsung tertangani secara otomatis.



### 3. Soal Unguided 3

- 3) Diberikan  $n$  data integer positif dalam keadaan terurut membesar dan sebuah integer lain  $k$ , apakah bilangan  $k$  tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu  $n$  dan  $k$ .  $n$  menyatakan banyaknya data, dimana  $1 < n \leq 1000000$ .  $k$  adalah bilangan yang ingin dicari.

Baris kedua berisi  $n$  buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari ( $k$ ) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data  $k$  tersebut tidak ditemukan dalam kumpulan.

Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan berikut ini.

```
package main
import "fmt"

const NMAX = 1000000
var data [NMAX]int

func main(){
/* buatlah kode utama yang membaca baris pertama (n dan k). kemudian data
diisi
oleh prosedur isiArray(n), dan pencarian oleh fungsi posisi(n,k), dan
setelah
itu output dicetak. */
}
```

Jawaban :

```
package main

import "fmt"

const NMAX = 1000000
```

```
var data [NMAX]int

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    kr := 0
    kn := n - 1
    found := -1

    for kr <= kn && found == -1 {
        med := (kr + kn) / 2

        if data[med] < k {
            kr = med + 1
        } else if data[med] > k {
            kn = med - 1
        } else {
            found = med
        }
    }

    return found
}
```

```

}

func main() {

    var n, k int

    fmt.Scan(&n, &k)

    isiArray(n)

    idx := posisi(n, k)

    if idx == -1 {

        fmt.Println("TIDAK ADA")

    } else {

        fmt.Println(idx)

    }

}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_11\Unguided\Unguided-3
$ go run .\implementbiner.go
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_11\Unguided\Unguided-3
$ go run .\implementbiner.go
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA

```

Penjelasan :

Program ini merupakan solusi pencarian elemen tunggal  $k$  pada rentang elemen *array*  $n$  yang ukurannya sangat besar (hingga 1.000.000 data) menggunakan arsitektur subprogram. Karena input data sudah dijamin terurut membesar (*ascending*) dan memiliki limitasi data yang masif, program ini secara spesifik menggunakan fungsi *Binary Search* agar proses pencariannya optimal secara memori dan waktu eksekusi. Fungsi  $posisi(n, k)$  akan mengembalikan posisi indeks data  $k$  jika ditemukan sesuai logika biner, atau mem-*passing* nilai -1 ke fungsi utama untuk mencetak keluaran "TIDAK ADA" jika data  $k$  tidak ada di kumpulan *array*.

#### D. Kesimpulan

Berdasarkan serangkaian praktikum pada Modul 12, dapat disimpulkan bahwa pemilihan algoritma *searching* sangat bergantung pada karakteristik dataset yang kita kelola. Algoritma *Sequential Search* sangat mudah diimplementasikan dan andal digunakan untuk dataset berukuran kecil maupun data yang terdistribusi secara acak tanpa perlu diurutkan. Namun, ketika berhadapan dengan dataset berskala besar, algoritma *Binary Search* menjadi solusi yang jauh lebih optimal dan efisien dari segi performa waktu eksekusi, dengan kompensasi (syarat) bahwa sekumpulan data tersebut wajib melalui proses pengurutan (*sorting*) terlebih dahulu sebelum pencarian bisa dilakukan.